Projet 3

Réalisé par : Abderrahim YACHKOUR

Mise en place du projet

Fork du repository GitHub

Pour récupérer le projet fourni par OpenClassrooms, j'ai commencé par effectuer un fork du repository GitHub sur mon compte personnel. Cela m'a permis d'avoir ma propre copie du projet sur laquelle travailler librement sans modifier le repository original.

Création d'un nouveau projet Salesforce dans VS Code

Le projet forké depuis GitHub ne contenait pas la structure complète nécessaire pour développer avec Salesforce CLI et VS Code. Il contenait uniquement les dossiers essentiels du code source (classes, flexipages, lwc) sans la structure force-app/main/default qui est indispensable pour déployer sur une org Salesforce.
Pour résoudre cela, j'ai créé un nouveau projet Salesforce directement depuis VS Code (via Salesforce Extension Pack), ce qui a généré automatiquement toute la structure nécessaire : force-app/main/default avec les dossiers applications, aura, classes, contentassets, flexipages, layouts, lwc, objects, permissionsets, staticresources, tabs, triggers, ainsi que les fichiers de configuration (.sf, .sfdx, .husky, sfdx-project.json, etc.).

Intégration des fichiers du projet forké

Une fois le nouveau projet Salesforce créé, j'ai copié les dossiers du projet forké (classes, flexipages, lwc) et je les ai collés dans le répertoire force-app/main/default du nouveau projet. Cela a permis d'avoir une structure complète et fonctionnelle, combinant la génération automatique de VS Code avec les fichiers sources fournis par OpenClassrooms.

Premier Exercice : Améliorez une interface d'une organisation Salesforce en utilisant les composants Web Lightning

accountOpportunitiesViewer

Fichier HTML — Bug 1

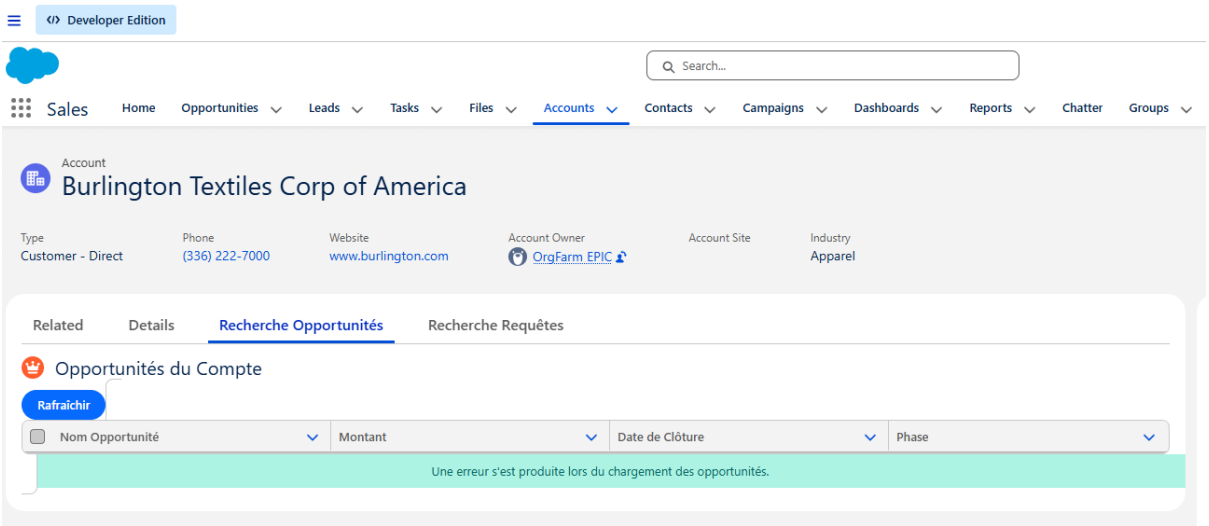
Erreur : <template if:false={opportunities}> affichait le tableau quand il n'y avait PAS de données.

Pour corriger cette erreur il faut remplacer le if:false par if:true.

Avant : `<template if:false={opportunities}>`

Après : `<template if:true={opportunities}>`

Fichier HTML — Bug 2



Erreur : la classe slds-theme_success verte utilisée sur le message d'erreur.

Pour corriger cette erreur il faut remplacer slds-theme_success par slds-theme_error.

Avant :

```
<template if:true={error}>
                <div class="slds-notify slds-notify_alert slds-theme_alert-
texture slds-theme_success" role="contentinfo">
                    <span class="slds-assistive-text">Erreur</span>
                    <h2>Une erreur s'est produite lors du chargement des
opportunités.</h2>
                </div>
</template>
```

Après :

```
<template if:true={error}>
                <div class="slds-notify slds-notify_alert slds-theme_alert-
texture slds-theme_error" role="contentinfo">
                    <span class="slds-assistive-text">Erreur</span>
                    <h2>Une erreur s'est produite lors du chargement des
opportunités.</h2>
                </div>
</template>
```

Fichier JS — Bug 3

Erreur : @wire utilisait $accountId alors que la propriété déclarée s'appelle recordId.

Correction : remplacer $accountId par '$recordId'.

Avant :

```
@wire(getOpportunities, { recordId: '$accountId' }) //error
```

Après :

```
@wire(getOpportunities, { accountId: '$recordId' })
```

Fichier JS — Bug 4

Erreur : error = {} est un objet vide considéré comme vrai en JavaScript. Le message d'erreur s'affichait en permanence même sans erreur réelle.

Correction : suppression de l'initialisation pour laisser error à undefined.

Avant :

```
@track error = {};
```

Après :

```
@track error;
```

accountCasesSearcher

Fichier HTML — Ajout
Problème : quand la recherche ne trouvait aucun résultat, l'utilisateur voyait une page vide sans aucun message.
Correction : ajout d'un bloc pour afficher un message clair à l'utilisateur.

Ajout :

```
<template if:true={noResults}>
            <p class="slds-p-around_medium">Aucun cas trouvé pour cette
recherche.</p>
</template>
```

Fichier JS — Ajout
Correction : ajout d'une propriété calculée noResults qui retourne vrai quand le tableau de résultats est vide.

Ajout :

```
get noResults() {
    return this.cases && this.cases.length === 0;
}
```

Second Exercice : Codez une recherche de données dans un back-end Salesforce via Apex

AccountCasesController

Fichier Apex — Bug principal

Erreur : la méthode findCasesBySubject était complètement vide. Elle ne prenait aucun paramètre et ne retournait rien.

Correction : ajout des paramètres accountId et subjectSearchTerm, et de la requête SOQL pour chercher les cas par sujet.

Avant :

```
 @AuraEnabled
    public static void findCasesBySubject(){}
```

Après :

```
public static List<Case> findCasesBySubject(Id accountId, String
subjectSearchTerm) {
        String searchKey = '%' + subjectSearchTerm + '%';
        return [
            SELECT Id, Subject, Status, Priority
            FROM Case
            WHERE AccountId = :accountId
            AND Subject LIKE :searchKey
        ];
      }
```

Fichier Apex — Amélioration : pattern Selector

Problème : la requête SOQL était écrite directement dans le Controller, ce qui mélange deux responsabilités différentes. Le projet suit le pattern Selector : le Controller gère la communication avec le LWC, et le Selector gère les requêtes SOQL.

Correction : création d'une classe CaseSelector séparée qui contient uniquement la requête SOQL, puis mise à jour du Controller pour l'appeler.

Nouvelle classe CaseSelector.cls :

```
public with sharing class CaseSelector {

    public static List<Case> getCaseByAccountId (Id accountId,String
subjectSearchTerm)
 {
             String searchkey='%'+ subjectSearchTerm+'%';
             return [SELECT Id, Subject, Status, Priority FROM Case WHERE
AccountId = :accountId AND Subject LIKE :searchkey];
 }
}
```

AccountCasesController mis à jour :

```
Public with sharing class AccountCasesController {
@AuraEnabled(cacheable=true)
public static List<Case> findCasesBySubject(Id accountId, String
subjectSearchTerm) {
return CaseSelector.getCaseByAccountId(accountId, subjectSearchTerm);}}
```

Ce pattern améliore la maintenabilité : si la requête SOQL doit évoluer, elle n'est modifiée qu'à un seul endroit dans CaseSelector, sans toucher au Controller.

Concepts Apex utilisés

with sharing

Le mot-clé with sharing indique que la classe respecte les règles de partage de données de l'org Salesforce. Un utilisateur ne voit que les enregistrements auxquels il a accès, ce qui garantit la sécurité des données.

@AuraEnabled

L'annotation @AuraEnabled rend une méthode Apex accessible depuis un composant LWC. Sans cette annotation, le composant front-end ne peut pas appeler la méthode, même si elle est publique.

Opérateur LIKE et '%'

En SOQL, l'opérateur LIKE permet de rechercher un mot contenu dans une chaîne. Le symbole % signifie "n'importe quels caractères". Encadrer le terme de recherche avec % de chaque côté permet de trouver le mot n'importe où dans le champ.

Exemple : searchKey = '%livraison%' retourne tous les Cases dont le sujet contient le mot livraison.

Conclusion

Ce projet m'a permis de travailler sur un cas concret de développement Salesforce, de la mise en place de l'environnement jusqu'à la correction et l'amélioration du code.
La première étape était la mise en place du projet : fork du repository GitHub, création d'un nouveau projet Salesforce dans VS Code pour obtenir la structure force-app/main/default, puis intégration des fichiers sources fournis. Cela m'a appris que la structure d'un projet Salesforce ne se résume pas au code — elle est indispensable pour que le déploiement fonctionne.
Sur le premier exercice, j'ai corrigé quatre bugs dans le composant accountOpportunitiesViewer (erreurs de directives if:true/if:false, mauvaise classe CSS, mauvais nom de propriété dans le @wire, initialisation incorrecte de error) et j'ai amélioré accountCasesSearcher en ajoutant une gestion des résultats vides. J'ai compris comment les fichiers HTML, JS et Apex sont liés dans un composant LWC, et à quel point une petite erreur dans un fichier peut bloquer tout l'affichage.
Sur le second exercice, j'ai corrigé une méthode Apex vide et implémenté une vraie requête SOQL avec les paramètres accountId et subjectSearchTerm. J'ai ensuite appliqué le pattern Selector en séparant la logique de requête dans une classe CaseSelector indépendante. Cela m'a appris l'importance de la séparation des responsabilités : le Controller communique avec le LWC, le Selector gère la base de données.
En résumé, ce projet m'a donné une vision complète du cycle de développement Salesforce : mise en place de l'environnement, débogage LWC, écriture et optimisation de requêtes SOQL, et structuration du code Apex selon les bonnes pratiques.

Voici le lien vers le repository où j'ai déposé les fichiers du code pour cette organisation.
https://github.com/AbderrahimYachkour/Projet-3---EG-Manufacturing.git